One to Many Relationship

💡 **Rule of Thumb:**

> In one-to-many relationships, the reference almost always goes in the “many” side.

🟢 One-to-Many (1:N) Relationship

👉 In a **one-to-many relationship**,

- A single record in **Table A** can be linked to **many records** in **Table B**,
- But each record in **Table B** is linked to **only one record** in **Table A**.

🔹 Real-World Examples

- One **customer** can place many **orders**.
- One **teacher** can teach many **students**.
- One **department** can have many **employees**.

🔹 SQL Example (Relational DB)

```sql
-- Customers table
CREATE TABLE customers (
    customer_id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(100) NOT NULL
);

-- Orders table (Many orders for one customer)
CREATE TABLE orders (
    order_id INT PRIMARY KEY AUTO_INCREMENT,
    order_date DATE NOT NULL,
```

```
    customer_id INT,
    FOREIGN KEY (customer_id) REFERENCES customers(customer_i
d)
);
```

👉 Here:

- `customers` → parent (one).
- `orders` → child (many).
- Each order references **exactly one** customer.

🔹 MongoDB Example (NoSQL, Mongoose)

1️⃣ Referencing (Common Approach)

```
// user.model.js
const userSchema = new mongoose.Schema({
  name: String
});

// order.model.js
const orderSchema = new mongoose.Schema({
  orderDate: Date,
  customer: { type: mongoose.Schema.Types.ObjectId, ref: "Use
r" } // many orders → one user
});
```

🔹 ERD Symbol (Crow's Foot Notation)

```
[Customer] |——< [Order]
```

- `|` → one
- `<` (crow's foot) → many

🟢 One-to-Many in MongoDB

👉 In MongoDB, a **collection** is like a table.

- **Collection A** = parent (the "one")
- **Collection B** = child (the "many")

So:

- One document in **Collection A** can be linked to many documents in **Collection B**.
- But each document in **Collection B** refers back to only one document in **Collection A**.

🔹 Example: Customer → Orders

Option 1: Referencing (Normalized approach)

👉 Store **many order documents** in an `orders` collection, each with a reference to one customer.

```
// customers collection
{
  _id: ObjectId("cust123"),
  name: "John Doe",
  email: "john@example.com"
}

// orders collection (many orders for one customer)
{
```

```
  _id: ObjectId("ord1"),
  orderDate: "2025-09-24",
  amount: 500,
  customer: ObjectId("cust123")   // reference back to custom
er
}
{
  _id: ObjectId("ord2"),
  orderDate: "2025-09-25",
  amount: 300,
  customer: ObjectId("cust123")
}
```

- One **customer** → many **orders**.
- Each **order** has only one `customer` reference.

📌 Mongoose Schema:

```
const customerSchema = new mongoose.Schema({
  name: String,
  email: String
});

const orderSchema = new mongoose.Schema({
  orderDate: Date,
  amount: Number,
  customer: { type: mongoose.Schema.Types.ObjectId, ref: "Cus
tomer" }
});
```

💡 **Rule of Thumb:**

> In one-to-many relationships, the reference almost always goes in the “many” side.

✅ 15 Real-Life 1-to-Many Relationship Use Cases (MongoDB + Mongoose)

📌 1-to-Many Relationships with Arrows (`child --> parent`)

#	Parent (One)	Child (Many)	Reference Direction
1	User	Orders	`orders --> users`
2	User	Addresses	`addresses --> users`
3	BlogPost	Comments	`comments --> blogposts`
4	Product	Reviews	`reviews --> products`
5	Teacher	Students	`students --> teachers`
6	Course	Lessons	`lessons --> courses`
7	Invoice	Payments	`payments --> invoices`
8	Hospital	Doctors	`doctors --> hospitals`
9	Doctor	Appointments	`appointments --> doctors`
10	Student	Assignments	`assignments --> students`
11	Category	Products	`products --> categories`
12	Company	Employees	`employees --> companies`
13	Event	Attendees	`attendees --> events`
14	Library	Books	`books --> libraries`
15	Seller (Vendor)	Products	`products --> sellers`

🟢 1. Define Mongoose Schemas

Customer Schema (Parent)

```
const mongoose = require("mongoose");

const customerSchema = new mongoose.Schema({
  name: { type: String, required: true },
  email: { type: String, required: true, unique: true }
});

const Customer = mongoose.model("Customer", customerSchema);
module.exports = Customer;
```

Order Schema (Child, stores reference to parent)

```
const orderSchema = new mongoose.Schema({
  orderDate: { type: Date, default: Date.now },
  amount: Number,
  customer: { type: mongoose.Schema.Types.ObjectId, ref: "Cus
tomer", required: true }
});

const Order = mongoose.model("Order", orderSchema);
module.exports = Order;
```

> Rule of Thumb: In 1:N, the child stores the reference to the parent.

🟢 2. CRUD Operations

1️⃣ Create a Customer

```
const customer = await Customer.create({
  name: "John Doe",
  email: "john@example.com"
```

```
});
```

2️⃣ Create Orders for that Customer

```
// order 1
const order1 = await Order.create({
  amount: 500,
  customer: customer._id
});

// order 2
const order2 = await Order.create({
  amount: 300,
  customer: customer._id
});
```

3️⃣ Read Orders with Customer Details

```
const orders = await Order.find({ customer: customer._id }).p
opulate("customer");
console.log(orders);
```

- `.populate("customer")` fetches **customer details** inside each order.

4️⃣ Read Customer with All Orders

```
const customerWithOrders = await Customer.findById(customer._
id);
const orders = await Order.find({ customer: customerWithOrder
s._id });
```

```
console.log({ customer: customerWithOrders, orders });
```

> Optional: You can also embed orders in customer, but referencing is better if orders grow large.

5️⃣ Update an Order

```
await Order.findByIdAndUpdate(order1._id, { amount: 600 });
```

6️⃣ Delete an Order

```
await Order.findByIdAndDelete(order2._id);
```

7️⃣ Delete a Customer and Cascade Delete Orders (Optional)

```
await Order.deleteMany({ customer: customer._id });
await Customer.findByIdAndDelete(customer._id);
```

```
const orders = await Order.find({ customer: customer._id }).p
opulate("customer");
```

🔹 🔑 Why `.populate()` Required what it does

🔹 Step 1: Without `.populate()`

- Jab aap `Order.find({ customer: customer._id })` karte ho, Mongoose sirf **child document** laata hai.
- `customer` field **sirf ObjectId** ke form me milega (reference), actual customer ka data nahi.

Example output without populate:

```
[
  {
    "_id": "order1",
    "orderDate": "2025-09-24",
    "amount": 500,
    "customer": "64f0c5c8abc123456789abcd"   // sirf ID
  }
]
```

- Yaha, `customer` ka naam, email ya details nahi milega. Sirf **ID** hai.

🔹 Step 2: With `.populate("customer")`

- `.populate("customer")` bolta hai Mongoose ko:

> "Hey Mongoose, jo customer field me ObjectId hai, us ID ka actual document load karke replace kar do."

Example output with populate:

```
[
  {
    "_id": "order1",
    "orderDate": "2025-09-24",
    "amount": 500,
    "customer": {
        "_id": "64f0c5c8abc123456789abcd",
        "name": "John Doe",
        "email": "john@example.com"
    }
  }
]
```

- Ab hum order ke sath **full customer info** bhi le sakte hain.
- Matlab ek query me **child + parent ka data dono** mil gaya.

🔹 🔑 Why `.populate()` Required

1. MongoDB me **reference store hota hai**, data embed nahi hota.
2. Agar parent ka actual data chahiye → `.populate()` use karte hain.
3. Avoids **manual second query**. Ek hi query me join type result mil jata hai.
4. Especially 1:N ya N:1 relationships me **child ke sath parent info laane ke liye** must hai.